



Informatics II

Exercise 4

Grgic Anto

Schedule (Corrected)

Date	Topic
27. Feb	C environment
06. Mar	Ex1(Introduction to C, basic sorting)
13. Mar	Ex2 (Recursion)
20. Mar	Ex3 (Complexity, Correctness)
27. Mar	Ex4 (Divide and Conquer, Recurrence)
03. Apr	NO LAB (Please attend another class) – Spring break
17. Apr	Ex6 (Pointers, Linked Lists)
24. Apr	Ex7(ADTs, Stacks, Queues)
01. May	NO LAB (Please attend another class) – Labor day
08. May	Ex9 (Hash Tables)
15. May	Ex10 (Dynamic Programming)
22. May	Ex11 (Graphs)
29. May	Recap
04. Jun	Exam at Messe 16:00-19:00

***NEXT WEEK THIS EXERCISE CLASS WON'T TAKE PLACE.**

Divide and Conquer Algorithms

- **Basic Idea:**

- Divide the main problem in small, more manageable problems
- Solve the smaller problems
- Combine the solutions of the smaller problems into one for the big problem

- **Uses recursion as it's mainframe**

- **(Asymptotically) faster than the “Brute force” counterparts**

- But requires more memory space (For the recursive calls)

- **Time complexity can be calculated with it's recursive relation**

Recursive relation

- Describe the exact runtime of one call
 - The exact runtime has a recursive component, calling itself for another input: $T(n) = 32 \cdot T(n-1) + f(n)$
- Asymptotic complexity can be computed with:
 - Repeated Substitution
 - Substitution Method
 - Recursion Tree
 - Master Method

Repeated Substitution

- Not strictly formal but straight forward
- Substitute, and expand until a pattern emerges
 - The pattern will depend on another variable (ex. k)
- Set your variable so that the recursive relation for $T(n)$ calls the base case
- Get the asymptotical complexity

Recurrence :

$$T(n) = 2 \cdot T(n - 1) + 1, \quad T(0) = c$$

Expansion :

$$\begin{aligned} T(n) &= 2 \cdot T(n - 1) + 1 \\ &= 2 [2 \cdot T(n - 2) + 1] + 1 = 4 \cdot T(n - 2) + 2 + 1 \\ &= 4 [2 \cdot T(n - 3) + 1] + 2 + 1 = 8 \cdot T(n - 3) + 4 + 2 + 1 \end{aligned}$$

General pattern after k steps :

$$T(n) = 2^k \cdot T(n - k) + \sum_{i=0}^{k-1} 2^i = 2^k \cdot T(n - k) + (2^k - 1)$$

Set $k = n$:

$$T(n) = 2^n \cdot c + 2^n - 1 = (c + 1) \cdot 2^n - 1$$

Result : $T(n) \in \mathcal{O}(2^n)$

Substitution Method

- More formal, but requires a good guess
- Guess the form of the solution.
- Use induction to prove the solution
- Get the asymptotical complexity

► **Show that** $T(n) \leq cn^3$ **for** $T(n) = 4T(n/2) + n$

Guess : $T(n) \leq cn^3$

$$\begin{aligned} T(n) &= 4T(n/2) + n && \text{(recurrence)} \\ &\leq 4c(n/2)^3 + n && \text{(inductive hyp.)} \\ &= \frac{cn^3}{2} + n && \text{(simplification)} \\ &= cn^3 - \left(\frac{cn^3}{2} - n\right) && \text{(regrouping)} \\ &\leq cn^3 && \text{(holds when } cn^2 \geq 2) \end{aligned}$$

Result : $T(n) \leq cn^3 \in \mathcal{O}(n^3)$

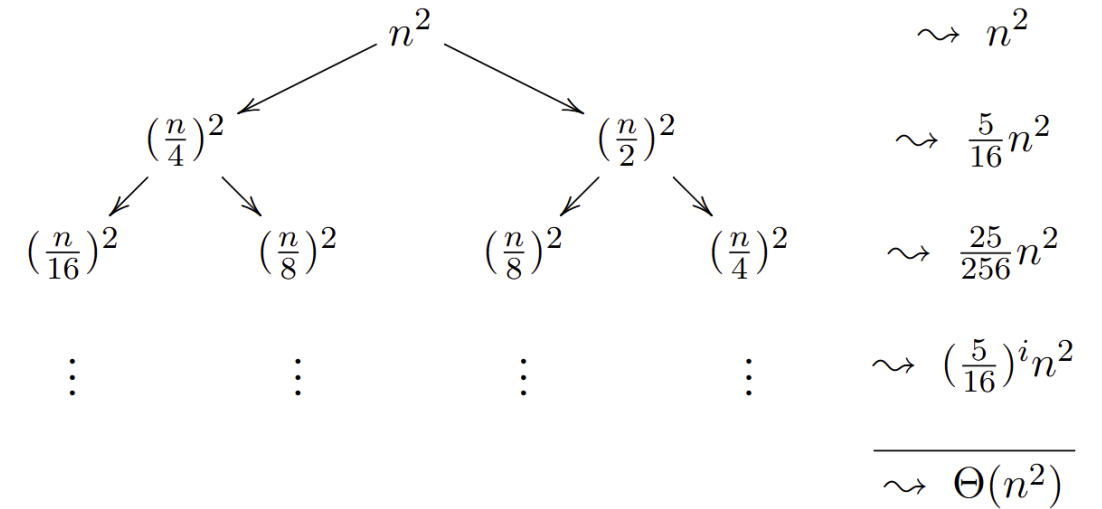
***Note that here we are getting a upper bound. We could use the same process to find a lower bound (es. a constant would be trivial but still correct.)**

To find a tight bound we would need to find an upper bound and a lower bound of the same asymptotic complexity.

Recursion Tree

- Similar to the repeated substitution but graphical
- Get the basic cost ($f(n)$)
- Expand the tree and add each level cost
- Find a pattern and calculate the asymptotic complexity

$$T(n) = T(n/4) + T(n/2) + n^2$$



Master method

- The recurrence relation has to be in a specific form:

- Extract a , b and $f(n)$

- Check each case and simply insert the numbers:

$$T(n) = a T(n/b) + f(n)$$

$a \geq 1$: number of subproblems

$b > 1$: factor by which input shrinks

$f(n)$: cost of divide/combine step

Case 1 : Leaves dominate

If $f(n) \in \mathcal{O}(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$

Then $T(n) \in \Theta(n^{\log_b a})$

Example : $T(n) = 8T(n/2) + n^2$

$\log_b a = \log_2 8 = 3$

$f(n) = n^2 \in \mathcal{O}(n^{3-1})$ ✓

$\Rightarrow T(n) \in \Theta(n^3)$

Case 2 : Equal split

If $f(n) \in \Theta(n^{\log_b a})$

Then $T(n) \in \Theta(n^{\log_b a} \cdot \log n)$

Example : $T(n) = 2T(n/2) + n$

$\log_b a = \log_2 2 = 1, \quad f(n) = n \in \Theta(n^1)$ ✓

$\Rightarrow T(n) \in \Theta(n \log n)$

Case 3 : Root dominates

If $f(n) \in \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$

and $a \cdot f(n/b) \leq k \cdot f(n)$ for some $k < 1$
(regularity condition)

Then $T(n) \in \Theta(f(n))$

Example : $T(n) = 3T(n/4) + n^2$

$\log_b a = \log_4 3 \approx 0.79$

$f(n) = n^2 \in \Omega(n^{0.79 + \varepsilon})$ ✓

Regularity : $a \cdot f(n/b) = 3 \cdot (n/4)^2 = \frac{3}{16} n^2 \leq k \cdot n^2$

with $k = \frac{3}{16} < 1$ ✓

$\Rightarrow T(n) \in \Theta(n^2)$

Thank you for your attention.

**If you have any questions write in the Olat forum
or come up to me.**